

Problem Set 2

BM2043 - Algorithms and Data Structure Lab

September 17, 2024

Stacks and Linked List

1. Implementation of Stack (Without STL)

Objective: Implement a stack data structure from scratch without using the Standard Template Library (STL). Your stack class should support basic stack operations: `push`, `pop`, `top`, `isEmpty`, and `size`.

Instructions: Define a `Stack` class with the following methods:

- `push(int value)`: Adds an integer `value` to the top of the stack.
- `pop()`: Removes the top element from the stack and returns it. If the stack is empty, throw an exception or handle the underflow condition.
- `top()`: Returns the top element of the stack without removing it. If the stack is empty, throw an exception or handle the underflow condition.
- `isEmpty()`: Returns `true` if the stack is empty, otherwise `false`.
- `size()`: Returns the stack size.

Implement the stack using a fixed-size array to store the elements. Include a constructor to initialize the stack with a given capacity. Handle edge cases such as attempting to pop from an empty stack or pushing onto a full stack.

```
class Stack {
private:
    // Data members to store stack elements and the current size

public:
    // Constructor
    // Destructor
    // Push operation
    // Pop operation
    // Top operation
    // Check if stack is empty
    // Size
};
```

2. Implementation of Deque (Without STL)

Objective: Implement a deque (double-ended queue) data structure from scratch without using the Standard Template Library (STL). Your deque class should support basic deque operations: `pushFront`, `pushBack`, `popFront`, `popBack`, `front`, `back`, `isEmpty`, and `size`.

Instructions: Define a `Deque` class with the following methods:

- `pushFront(int value)`: Adds an integer `value` to the front of the deque.

- **pushBack(int value):** Adds an integer **value** to the back of the deque.
- **popFront():** Removes the front element from the deque and returns it. If the deque is empty, throw an exception or handle the underflow condition.
- **popBack():** Removes the back element from the deque and returns it. If the deque is empty, throw an exception or handle the underflow condition.
- **front():** Returns the front element of the deque without removing it. If the deque is empty, throw an exception or handle the underflow condition.
- **back():** Returns the back element of the deque without removing it. If the deque is empty, throw an exception or handle the underflow condition.
- **isEmpty():** Returns **true** if the deque is empty, otherwise **false**.
- **size():** Returns the deque size.

Implement the deque using a circular buffer (fixed-size array) to store the elements. Include a constructor to initialize the deque with a given capacity. Handle edge cases such as attempting to pop from an empty deque or pushing onto a full deque.

```
class Deque {
private:
    // Data members to store dequeue elements and the current size, front, back
public:
    // Constructor
    // Destructor
    // Push front operation
    // Push back operation
    // Pop front operation
    // Pop back operation
    // Front operation
    // Back operation
    // Check if deque is empty
    // Size
};
```

Problems related to Stacks and Linked Lists

Problem 1 (Valid Parentheses)

In the world of coding and compilers, Riya is working on a program that checks if the syntax of parentheses in a string is valid. A string containing just the characters '(', ')', '{', '}', '[', and ']' is said to be valid if:

1. Every opening bracket has a corresponding closing bracket of the same type.
2. The brackets are closed in the correct order.

Help Riya by writing a function that checks if the input string of parentheses is valid.

Examples:

Example 1:

- **Input:** `s = "()"`
- **Output:** `true`
- **Explanation:** The string has correctly closed parentheses.

Example 2:

- **Input:** `s = "()[]{}"`
- **Output:** `true`
- **Explanation:** The string contains valid combinations of different types of parentheses.

Example 3:

- **Input:** `s = "()["`
- **Output:** `false`
- **Explanation:** The parentheses are of mismatched types.

Example 4:

- **Input:** `s = "([])"`
- **Output:** `true`
- **Explanation:** The parentheses are correctly nested and closed.

Constraints:

- $1 \leq s.length \leq 10^4$
- The string consists only of the characters `'()[]{}'`.

Your Task: Write a function `isValid` that takes a string `s` as input and returns `true` if the string is valid, and `false` otherwise.

Class Definition:

```
class Solution {
public:
    bool isValid(string s) {
        // code here
    }
};
```

Problem 2 (Next Greater Element)

Pritesh is analyzing weather patterns using data collected from different sensors. Each sensor reports temperature readings as an array of integers, and the readings are compared to find when the next spike in temperature occurs. Given two distinct integer arrays `temps1` and `temps2`, where `temps1` is a subset of `temps2`, for each element in `temps1`, Pritesh wants to determine the next higher temperature reported in `temps2`. If no such spike occurs, return `-1`.

Help Pritesh determine the next greater element in `temps2` for each value in `temps1`.

Examples:

Example 1:

- **Input:** `temps1 = [4,1,2]`, `temps2 = [1,3,4,2]`
- **Output:** `[-1, 3, -1]`
- **Explanation:**
 - The next greater element for 4 in `temps2 = [1, 3, 4, 2]` is -1 (no next greater element).
 - The next greater element for 1 is 3.
 - The next greater element for 2 is -1 (no next greater element).

Example 2:

- **Input:** `temps1 = [2, 4]`, `temps2 = [1, 2, 3, 4]`
- **Output:** `[3, -1]`
- **Explanation:**
 - The next greater element for 2 in `temps2 = [1, 2, 3, 4]` is 3.
 - The next greater element for 4 is -1 (no next greater element).

Constraints:

- $1 \leq \text{temps1.length} \leq \text{temps2.length} \leq 1000$
- $0 \leq \text{temps1}[i], \text{temps2}[i] \leq 10^4$
- All integers in `temps1` and `temps2` are unique.
- All integers of `temps1` also appear in `temps2`.

Your Task: You do not need to read input or print anything. Complete the function `nextGreaterElement` which takes two integer arrays as input and returns an array where for each element in `temps1`, you find the next greater element in `temps2`.

Class Definition:

```
class Solution {
public:
    vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
        // Implement logic here to solve the problem
    }
};
```

Problem 3 (Remove Middle Node from Linked List)

Pritesh is working on a project involving linked lists, where he needs to process and modify the list based on specific criteria. One task is to remove the middle node of a linked list.

Given the head of a singly linked list, help Pritesh delete the middle node and return the head of the modified linked list. The middle node is defined as the $\lfloor n/2 \rfloor$ th node from the start using 0-based indexing, where $\lfloor x \rfloor$ denotes the largest integer less than or equal to x .

For example:

- For a list of size 7, the middle node is at index 3.
- For a list of size 4, the middle node is at index 2.

Examples:

Example 1:

- **Input:** head = [1, 3, 4, 7, 1, 2, 6]
- **Output:** [1, 3, 4, 1, 2, 6]
- **Explanation:** The list size is 7, so the middle node is at index 3 with value 7. After removing this node, the list becomes [1, 3, 4, 1, 2, 6].

Example 2:

- **Input:** head = [1, 2, 3, 4]
- **Output:** [1, 2, 4]
- **Explanation:** The list size is 4, so the middle node is at index 2 with value 3. After removing this node, the list becomes [1, 2, 4].

Example 3:

- **Input:** head = [2, 1]
- **Output:** [2]
- **Explanation:** The list size is 2, so the middle node is at index 1 with value 1. After removing this node, the list becomes [2].

Constraints:

- The number of nodes in the list is in the range $[1, 10^5]$.
- $1 \leq \text{Node.val} \leq 10^5$

Class Definition:

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
```

Your Task: You do not need to read input or print anything. Complete the function `deleteMiddle` which takes the head of a linked list as input and returns the head of the modified linked list.

Class Definition:

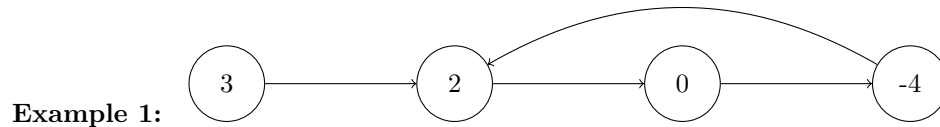
```
class Solution {
public:
    ListNode* deleteMiddle(ListNode* head) {
        // Implement logic to remove the middle node
    }
};
```

Problem 4 (Detect Cycle in a Linked List)

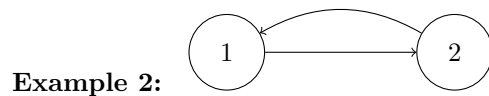
Nisarg is tackling a problem involving linked lists where he needs to identify the starting node of a cycle, if present, in the list. The task is to find the node where the cycle begins in a linked list. If there is no cycle, return `null`.

A linked list has a cycle if a node's `next` pointer eventually leads back to itself. The position of the cycle is represented by `pos`, where `pos` denotes the index of the node that the last node of the list points to. If `pos` is `-1`, there is no cycle.

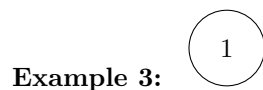
Examples:



- **Input:** `head = [3, 2, 0, -4]`, `pos = 1`
- **Output:** Node at index 1
- **Explanation:** The list has a cycle where the last node (`-4`) connects to the node with value 2, which is at index 1.



- **Input:** `head = [1, 2]`, `pos = 0`
- **Output:** Node at index 0
- **Explanation:** The list has a cycle where the last node (`2`) connects to the node with value 1, which is at index 0.



- **Input:** `head = [1]`, `pos = -1`
- **Output:** `null`
- **Explanation:** The list does not have a cycle.

Constraints:

- The number of nodes in the list is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- `pos` is `-1` or a valid index in the linked list.

Follow-up: Can you solve it using $O(1)$ (i.e., constant) memory?

Class Definition:

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
```

Your Task: You do not need to read input or print anything. Complete the function `detectCycle` which takes the head of a linked list as input and returns the node where the cycle begins, or `null` if there is no cycle.

Class Definition:

```
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        // Implement logic to detect the start of the cycle
    }
};
```

Problem 5 (Asteroid Collision)

Abhay is working on a problem involving asteroid collisions. Given an array `asteroids`, each integer represents an asteroid, where the absolute value is the asteroid's size, and the sign represents its direction (positive means moving to the right, and negative means moving to the left). The goal is to determine the state of the asteroids after all collisions.

Collision Rules:

- When two asteroids collide, the smaller one explodes.
- If both are the same size, both explode.
- Asteroids moving in the same direction do not collide.

Examples:

Example 1:

- **Input:** `asteroids = [5, 10, -5]`
- **Output:** `[5, 10]`
- **Explanation:** The 10 and -5 collide, resulting in 10. The 5 and 10 never collide.

Example 2:

- **Input:** `asteroids = [8, -8]`
- **Output:** `[]`
- **Explanation:** The 8 and -8 collide, resulting in both exploding.

Example 3:

- **Input:** asteroids = [10, 2, -5]
- **Output:** [10]
- **Explanation:** The 2 and -5 collide, resulting in -5. The 10 and -5 collide, resulting in 10.

Constraints:

- $2 \leq \text{asteroids.length} \leq 10^4$
- $-1000 \leq \text{asteroids}[i] \leq 1000$
- $\text{asteroids}[i] \neq 0$

Class Definition:

```
class Solution {
public:
    vector<int> asteroidCollision(vector<int>& asteroids) {
        // Implement logic to handle asteroid collisions
    }
};
```

Problem 6 (Largest Rectangle in Histogram)

Abhishek is solving a problem involving histograms. Given an array **heights** where each element represents the height of a bar in a histogram, and the width of each bar is 1, the task is to find the area of the largest rectangle that can be formed in the histogram.

Examples:**Example 1:**

- **Input:** heights = [2, 1, 5, 6, 2, 3]
- **Output:** 10
- **Explanation:** The largest rectangle has an area of 10, which spans the height 5 and 6.

Example 2:

- **Input:** heights = [2, 4]
- **Output:** 4
- **Explanation:** The largest rectangle has an area of 4, spanning the entire histogram.

Constraints:

- $1 \leq \text{heights.length} \leq 10^5$
- $0 \leq \text{heights}[i] \leq 10^4$

Class Definition:

```
class Solution {  
public:  
    int largestRectangleArea(vector<int>& heights) {  
        // Implement the logic to calculate the largest rectangle in histogram  
    }  
};
```